Lösungen und Hinweise für die Lehrkraft

Arbeitsblatt „Wie viele Stammbrüche braucht man?“

Ablauf

Phase	Zeit	Inhalt
Einstieg	5 min	Ägypten, Papyrus Rhind; Bild des Papyrus zeigen
Teil A	8 min	Stammbrüche von Hand
Teil B	12 min	Greedy-Rezept (Fibonacci), Überraschung 5/121
Teil C	12 min	Rekordtabelle, log-Taste als „Stellenzahl“
Teil D	8 min	Erdős, Zeitleiste, **der Wow-Moment**
(45 min)		
Teil E	30–45 min	Kritische Einordnung, Gruppendiskussion

Für 45 Minuten Teil E weglassen oder nur E1 kurz im Plenum besprechen.

Teil A

A1 1/3 + 1/15 = 5/15 + 1/15 = 6/15 = 2/5. 1/4 + 1/28 = 7/28 + 1/28 = 8/28 = 2/7.

A2 3/4 = 1/2 + 1/4; 5/6 = 1/2 + 1/3; 7/12 = 1/3 + 1/4 (auch 1/2 + 1/12).

Teil B

B1 3/7: Kehrwert 2,33 → 1/3; Rest 2/21. Kehrwert 10,5 → 1/11; Rest 1/231. **3/7 = 1/3 + 1/11 + 1/231.**

B2 10/11: 1/2, Rest 9/22; 1/3, Rest 5/66; 1/14, Rest 4/924 = 1/231. **10/11 = 1/2 + 1/3 + 1/14 + 1/231**, also **4** Stammbrüche. Kürzer geht es nicht (siehe Tabelle in Teil C); die kürzeste Zerlegung ist nicht eindeutig, z. B. auch 10/11 = 1/2 + 1/3 + 1/15 + 1/110.

B3 5 : 121 = 0,04132231…, und 1/25 + 1/759 + 1/208725 = 0,04132231… (stimmt exakt). Erkenntnis: Das Rezept funktioniert immer, liefert aber **nicht immer die kürzeste** Zerlegung, und die Nenner können **explodieren**. Weitere Beispiele: 4/17 liefert mit dem Rezept 4 Stammbrüche, kürzer ist 1/5 + 1/34 + 1/170; 77/79 liefert 7, kürzer geht es mit 6.

Teil C

C1 Die Zähler sind fast immer **Nenner minus 1**: Brüche **dicht bei 1** sind besonders schwer. (Grund: Man muss mit 1/2, 1/3, … anfangen, und der Rest bleibt lange groß.)

C2 Stellenzahlen 1, 1, 1, 2, 2, 2, 3, 5: Die Rekordnenner wachsen **immer schneller**, die Stellenzahl selbst wächst immer schneller (2 → 3 → 5).

C3

Stammbrüche	4	5	6	7	8
b	11	17	79	733	27 539
log(b)	1,04	1,23	1,90	2,87	4,44
log(log(b))	0,02	0,09	0,28	0,46	0,65

Ab 5 Stammbrüchen wächst log(log(b)) pro Stammbruch um etwa **0,19**. Anders gesagt: Pro „+1 bei der Stellenzahl der Stellenzahl“ kommen etwa 5 Stammbrüche dazu.

C4 Von 0,65 bis 2 sind es 1,35 ≈ 7 Schritte zu 0,19 → etwa **8 + 7 = 15** Stammbrüche für einen 100-stelligen Nenner. Betonen: **Schätzung**, nicht bewiesen. Genau hier setzt der Beweis an: Er garantiert, dass das Wachstum nie schneller als „ein festes Vielfaches von log(log(b))“ wird.

Herkunft der Tabelle: Die Rekordnenner 2, 3, 5, 11, 17, 79, 733, 27539 sind die OEIS-Folge A097048. Die Werte bis 733 wurden mit `optimum.py` aus diesem Repository unabhängig nachgerechnet (vollständige Suche).

Teil D

D1

Nenner	S	S / log(S)	√S	log(S)
100 Stellen	100	50	10	2
Googol	101	≈ 50,4	≈ 10,05	≈ 2,00
Googolplex	10^{100}	10^{98}	10^{50}	**100**

Beobachtung: Für riesige Nenner ist die neue Grenze winzig im Vergleich zu den alten. Selbst für einen Googolplex (eine Zahl, die man nicht einmal aufschreiben könnte, weil es nicht genug Atome im Universum für ihre Ziffern gibt) wächst die neue Grenze nur auf 100 (mal einen festen Faktor). *Hinweis:* Alle drei Grenzen gelten nur „bis auf einen festen Faktor“; welche Basis der Logarithmus hat, ändert nur diesen Faktor.

D2 Erdős’ Verfahren wächst gleichmäßig steil (bei 50 Stellen 78 Stammbrüche), das neue Verfahren bleibt fast flach (bei 50 Stellen 28). Bei 1000 Stellen braucht Erdős’ Verfahren 894 Stammbrüche (gemessen mit `erdos1950.py`); das neue Verfahren ist dort nicht mehr rechenbar, hochgerechnet etwa 45. Das neue Verfahren gewinnt also deutlich.

Teil E

E1 Es gibt **unendlich viele** Nenner. Ein Muster bei kleinen Zahlen kann später abbrechen. Nur ein Beweis deckt **alle** Nenner ab.

E2 Lean prüft jeden logischen Schritt. Mögliche Schwachstellen: (1) Die **Aussage** könnte in Lean anders formuliert sein als gemeint. Das prüfen Menschen; hier ist die Lean-Datei mit der Aussage nur etwa 30 Zeilen lang und gut lesbar (`lean/ComparatorChallenges/ShortEgyptianFractions.lean` im OpenAI-Repository). (2) Fehler in Lean selbst oder in den verwendeten Grundlagen sind sehr unwahrscheinlich. (3) Ein geprüfter Beweis ist noch nicht **verstanden**: Menschen wollen auch wissen, *warum* es stimmt.

E3 Kein Widerspruch: Der Satz sagt, dass man mit **höchstens** $c \cdot \log(\log(b))$ auskommt, mit einer festen, aber unbekannten und möglicherweise großen Zahl c. Das Verfahren im Beweis ist ein Werkzeug, um das zu zeigen, und kein Rezept für die kürzeste Zerlegung. „Es geht immer mit höchstens so vielen“ ist etwas anderes als „so findet man die beste Lösung“.

E4 Offene Diskussion. Mögliche Impulse:

- Das Modell baut auf Jahrhunderten menschlicher Mathematik auf (Erdős, Vose, Fourier-Analysis usw.). Im Paper werden diese Vorarbeiten zitiert.
- Hunderte Beweise auf einmal: Gutachten dauern normalerweise Monate. Computer-Prüfung (Lean) hilft, aber nur für einen Teil der Ergebnisse.
- Schule: Rechnen und Beweisen verstehen bleibt wichtig, um Ergebnisse beurteilen zu können.

Hintergrund (für die Lehrkraft)

- **Satz (OpenAI, 25.09.2026, „Short Egyptian fractions“):** Es gibt Konstanten $c_1, c_2 > 0$, so dass für alle großen b gilt: $c_1 \cdot \log\log b \le N(b) \le c_2 \cdot \log\log b$, wobei $N(b)$ die größte Mindestanzahl verschiedener Stammbrüche für einen Bruch a/b ist.
- **Kernidee:** Man wählt eine Hilfszahl M mit sehr vielen Teilern (Produkte von Primzahlen). Mit jedem Stammbruch kann man den Zähler auf einen Rest modulo einem passenden Teiler verkleinern. Weil es so viele Teiler gibt, ist ein sehr kleiner Rest dabei, und nach wenigen Schritten ist man fertig.
- **Quellen:** OpenAI-Repository github.com/openai/math, Preprint `preprints/Short-Egyptian-fractions-September-25-2026`, Lean-Formalisierung `lean/docs/025.md`; Erdős-Problem Nr. 304 auf erdosproblems.com; OEIS A097048.
- **Zum Vorführen** (optional, Python): im Hauptverzeichnis dieses Repositorys
  - `python3 optimum.py 77 79`: kürzeste Zerlegung,
  - `python3 stammbruch_loglog.py 77 79 --alle`: Zerlegung nach dem neuen Beweis,
  - `python3 erdos1950.py --stellen 10 50 1000`: Erdős’ Verfahren.